

Session 17 : TP — Véhicules (Raycast Vehicle)

Objectifs du TP

- Comprendre le modèle **Raycast Vehicle** — et pourquoi on ne simule **jamais** les roues comme des rigid bodies.
- **Phase A** : implémenter la suspension à la **main** — raycast + ressort + amortisseur, roue par roue.
- Piloter le véhicule : **accélération, freinage, direction** — au clavier.
- **Phase B** : remplacer la suspension maison par le **vehicle controller** de Rapier (DynamicRayCastVehicleController) — et comparer.
- Comprendre la **tenue de route** : grip, slip angle, et pourquoi une voiture dérape.
- **Tout réutiliser** : rigid bodies (S14), raycasts (S15), la philosophie du character controller (S16), les ressorts (S7).

💡 Le TP final

C'est la **convergence** du cours. Un véhicule, c'est un **character controller à roues** : le code décide (throttle, direction), les raycasts vérifient (où est le sol sous chaque roue ?). Entre les deux : une **suspension** — le ressort de la Session 7, piloté par le raycast de la Session 15, appliqué quatre fois par frame. Vous avez toutes les briques ; aujourd'hui, on les assemble.

Où vous écrivez — et ce qui est déjà fourni

Ouvrez `exemples/session17_vehicle.js`. La GUI bascule entre deux modes, et deux fonctions vous attendent :

1. `updateManual(dt)` — **Phase A** → vos **M2** (suspension) et **M3** (pilotage).
2. `updateRapier(dt)` — **Phase B** → votre **M5** (le vehicle controller).

Tout le reste est écrit : lisez-le, ne le réécrivez pas. `createVehicle()` (châssis + collider + modèle 3D + roues visuelles), `createTerrain()` (sol, murs, rampe, bosses), `updateVisuals()` (roues, caméra, HUD) et la boucle `animate()` (pas de temps fixe + `resetForces/resetTorques`).

Helpers fournis (en bas du fichier) :

- `castWheelRay(attach, maxDist)` — le raycast de suspension (il **exclut** le collider du châssis).
- `verticalVelocity(attach, pos, vel, angvel)` — vitesse verticale du point d'attache ($v + \omega \times r$).
- `chassisForward(quat)` — le vecteur « avant » du châssis.
- `applyGrip(quat, vel, groundPoint)` — l'adhérence latérale (donné).
- Les constantes `WHEELS[]`, `WHEELBASE` et l'objet `params` (les réglages de la GUI).

Commandes : W/↑ accélérer · S/↓ freiner-reculer · A/D (ou ←/→) tourner · R réinitialiser.

Partie 1 : Le modèle Raycast Vehicle

Pourquoi pas 4 roues rigid bodies ?

L'intuition du débutant : « une roue = un rigid body cylindrique + un joint revolute ». Ça **marche** — et c'est un cauchemar :

- 4 joints à résoudre à **chaque frame** — le solveur lutte entre roue et châssis (oscillations).
- La moindre bosse **décolle** une roue — le joint s'étire, la voiture part en vrille.
- Le moindre virage **retourne** le châssis — il n'y a rien pour le tenir.

Le modèle des jeux **arcade** : les roues ne sont **pas des corps**. Ce sont des **raycasts** — quatre lignes invisibles qui interrogent le sol.

Raycast Vehicle

- Un châssis : rigid body dynamique (boîte).

Quatre roues « virtuelles » : un point d'attache sur le châssis + un **raycast** vers le bas.

- Si le rayon touche le sol : la roue **porte** — on calcule la force de suspension et on l'applique au châssis.
- Si le rayon ne touche rien : la roue est **en l'air** — pas de force, la roue **pend**.

Avantages : stable, performant, et les paramètres (raideur, amortissement, grip) se rèlent **directement** — sans se battre contre le solveur.

🔗 Roues visuelles, roues physiques

Les roues que le joueur **voit** (meshes cylindriques) sont **découplées** des raycasts : chaque frame, on positionne le mesh de la roue au point d'impact du rayon (ou à la position « pendante » si en l'air), et on le fait tourner selon la vitesse. Le visuel **suit** la physique — il ne la **subit** pas.

Le **corps** de la voiture est un modèle 3D (Kenney, CC0) purement décoratif : le collider, lui, reste la boîte du châssis.

Partie 2 : Phase A — la suspension à la main

La boucle par roue

Pour chaque roue, à chaque frame physique :

1. **Raycast** depuis le point d'attache, vers le bas, de longueur $L_{\text{repos}} + r_{\text{roue}}$ — en **excluant** le collider du châssis.
2. Si on touche le sol à distance d : la **longueur de suspension** vaut $d - r_{\text{roue}}$, et la **compression** $c = 1 - \frac{d - r_{\text{roue}}}{L_{\text{repos}}}$ (0 = détendue, 1 = écrasée).
3. **Force de ressort** : $F_k = k \cdot c \cdot L_{\text{repos}}$ (le ressort de la S7).
4. **Amortisseur** : $F_c = c_{\text{damp}} \cdot v_{\text{suspension}}$ (la vitesse **verticale** du point d'attache).
5. On applique $F = F_k - F_c$ **au point d'attache** : `addForceAtPoint`.

```
// Phase A — à écrire dans updateManual()
// L'attache et le rayon sont DONNÉS ; vous écrivez le bloc M2.
for (let i = 0; i < 4; i++) {
    const w = WHEELS[i];
    const attach = w.attach.clone().applyQuaternion(quat)
        .add(new THREE.Vector3(pos.x, pos.y, pos.z)); // DONNÉ
    const hit = castWheelRay(attach, w.restLength + w.radius); // DONNÉ

    if (hit) {
        // === M2 — LA SUSPENSION (à écrire) ===
        const suspLength = Math.max(0, hit.timeOfImpact - w.radius);
        const compression = 1 - suspLength / w.restLength;
        const springF = params.k * compression * w.restLength;
        const damperF = params.damping *
            verticalVelocity(attach, pos, vel, angvel);
```

```

        chassisBody.addForceAtPoint(
            { x: 0, y: springF - damperF, z: 0 },
            { x: attach.x, y: attach.y, z: attach.z }, true);
    }
}

```

Les **roues visuelles** ne sont pas ici : `updateVisuals()` (donné) place chaque mesh au point d'impact du **même** rayon, et le fait rouler selon la vitesse.

💡 Valeurs de départ

Châssis : cuboïde (0.9, 0.25, 1.2) (demi-tailles), densité 150 (≈ 324 kg).

Par roue : $k = 12,000$ N/m, $c_{\text{damp}} = 800$ N dot s/m, $L_{\text{repos}} = 0.5\text{m}$, $r_{\text{roue}} = 0.35\text{m}$. Force moteur **totale** : 2500N (appliquée au sol).

Ces valeurs vivent dans l'objet `params`, et la **GUI** les expose en direct : `k`, `damping`, `grip`, `engineForce`, `steerMax`, `steerGain`. Réglez en roulant, pas dans le code.

Méthode de réglage : commencez sans amortisseur — la voiture **oscille** (c'est le symptôme).

Ajoutez `damping` jusqu'à ce qu'elle se stabilise en ≈ 1 rebond. Trop d'amortisseur = la voiture « plonge » et reste écrasée.

⚠️ Le piège : appliquer la force au centre de masse

`addForce` applique la force **au centre de masse**. Or une force qui passe par le centre de masse ne crée **aucun couple** (S3) : la voiture ne peut ni **piquer** (accél./freinage) ni **rouler** (virage) — la caisse glisse comme sur des rails.

Il faut appliquer les forces **horizontales** (moteur, grip) **au niveau du sol**, sous le centre de masse, avec `addForceAtPoint`. Le **bras de levier** vertical h crée alors le couple de tangage $\tau = F \cdot h$ et le couple de roulis. C'est le **moment** de la force (S3) en action — et c'est ce qui fait vivre la caisse.

⚠️ Le piège n°2 : les forces s'accumulent (déjà géré)

Rapier n'efface **pas** les forces ajoutées avec `addForce` / `addForceAtPoint` / `addTorque` après un `world.step()` — le nettoyage automatique a été **retiré** en 2022. Elles s'**accumulent** donc frame après frame : au bout d'une seconde, la force de suspension est gigantesque et la voiture **s'envole vers le ciel**.

Il faut les remettre à zéro **une fois par pas physique**, juste avant de réappliquer les vôtres :

```

chassisBody.resetForces(true);
chassisBody.resetTorques(true);

```

C'est déjà fait dans la boucle `animate()` fournie — vous n'avez rien à ajouter. Mais c'est un piège à connaître : c'est le seul endroit du cours où les forces ne sont pas **recalculées** de zéro à chaque frame, mais **accumulées**.

Partie 3 : Piloter

Accélération et direction

```
// Phase A – à écrire dans updateManual(), APRÈS la boucle des roues.  
// `groundPoint` (le point AU SOL sous le centre de masse) est DONNÉ.  
const forward = chassisForward(quat);  
const vFwd = forward.x * vel.x + forward.y * vel.y + forward.z * vel.z;  
  
// === M3 – LE PILOTAGE (à écrire) ===  
// 1. Accélérer / freiner (W / S) : poussée longitudinale AU SOL  
const throttle = (keys['KeyW'] || keys['ArrowUp'] ? 1 : 0)  
                 - (keys['KeyS'] || keys['ArrowDown'] ? 0.75 : 0);  
if (throttle !== 0) {  
    chassisBody.addForceAtPoint(  
        { x: forward.x * params.engineForce * throttle,  
          y: forward.y * params.engineForce * throttle,  
          z: forward.z * params.engineForce * throttle }, groundPoint, true);  
}  
  
// 2. Tourner (A / D) : servo de lacet cinématique (modèle du vélo)  
const steerAngle = steerInput * params.steerMax;  
const omegaTarget = (vFwd / WHEELBASE) * Math.tan(steerAngle);  
chassisBody.addTorque(  
    { x: 0, y: params.steerGain * (omegaTarget - angvel.y), z: 0 }, true);
```

steerInput (lissé depuis les touches dans animate()) est donné : servez-vous en tel quel. La **direction** n'applique pas un couple constant, mais impose le **taux de lacet cinématique** du modèle du vélo : $\omega_{\text{cible}} = (v/L) \cdot \tan(\delta)$. Il est proportionnel à la vitesse **signée** — le braquage s'**inverse** donc en marche arrière, comme une vraie voiture — et **borné**, ce qui empêche la toupie. Le **grip** latéral (applyGrip, donné), lui, est appliqué **au sol** : c'est ce qui fait **rouler** la caisse dans le virage, et déraiper à la limite d'adhérence.

Grip et slip angle

- **Force longitudinale** : accélération / freinage — le long du plan de la roue.
- **Force latérale** : le **grip** — perpendiculaire au plan de la roue. Sans elle, la voiture glisse comme sur la glace.

Slip angle : l'angle entre la direction **du pneu** et la direction **réelle** du mouvement.

- Petit angle : la voiture tourne **proprement**.
- Grand angle : la voiture **dérape** (drift) — le grip « lâche ».

Modèle arcade simple : à chaque roue en contact, appliquer une force latérale proportionnelle à la vitesse latérale **locale** de la roue, plafonnée par μ (le grip max). Un μ élevé = karting, un μ faible = rallye sur neige.

Partie 4 : Phase B — le vehicle controller de Rapier

💡 Pourquoi changer d'outil ?

Vous venez de **sentir** chaque paramètre de la suspension — combien d'itérations pour stabiliser c_{damp} , comment un k trop mou fait toquer le châssis. C'était le but de la Phase A. En production,

personne n'écrit cette boucle : le moteur la fournit. Rapier expose le **DynamicRayCastVehicleController** — le modèle Bullet (btRaycastVehicle), intégré.

Le contrôleur, en une douzaine de lignes

```
// Phase B – à écrire dans updateRapier() : on remplace TOUTE la Phase A.
// 1. Créer le contrôleur UNE SEULE FOIS (création paresseuse)
if (!vehicleController) {
    vehicleController = world.createVehicleController(chassisBody);
    for (let i = 0; i < 4; i++) {
        const w = WHEELS[i];
        vehicleController.addWheel(
            { x: w.attach.x, y: w.attach.y, z: w.attach.z }, // attache (locale)
            { x: 0, y: -1, z: 0 }, // direction du rayon
            { x: -1, y: 0, z: 0 }, // axe de la roue
            w.restLength, w.radius); // longueur au repos, rayon
        // Paramètres NORMALISÉS par la masse du châssis (≈ 324 kg)
        vehicleController.setWheelSuspensionStiffness(i, 24); // ≈ votre k
        vehicleController.setWheelSuspensionRelaxation(i, 2.3); // détente
        vehicleController.setWheelSuspensionCompression(i, 4.4); // compression
        vehicleController.setWheelFrictionSlip(i, 10.5); // ≈ votre mu
        vehicleController.setWheelMaxSuspensionForce(i, 30000);
    }
}

// 2. Chaque frame – pilotage par roue
const throttle = (keys['KeyW'] || keys['ArrowUp'] ? 1 : 0)
    - (keys['KeyS'] || keys['ArrowDown'] ? 0.75 : 0);
for (let i = 0; i < 4; i++) {
    if (WHEELS[i].front) vehicleController.setWheelSteering(i, steerInput * 0.4);
    else vehicleController.setWheelEngineForce(i, -throttle * params.engineForce /
2);
}

// 3. Mettre à jour le contrôleur (AVANT world.step())
vehicleController.updateVehicle(dt);
```

Lecture par roue : `wheelIsInContact(i)`, `wheelSuspensionLength(i)`, `wheelRotation(i)`, `currentVehicleSpeed` — pour le visuel et le HUD.

Deux pièges de la Phase B. (1) **Signe** : avec un axe de roue $-\hat{i}$, une force moteur **positive** pousse la voiture en **marche arrière**. D'où le $-$ devant `throttle`. (2) **Par roue** : `setWheelEngineForce` s'applique à **chaque roue motrice** — on divise donc la force **totale** par le nombre de roues motrices (2). Les deux phases poussant **au sol**, `params.engineForce` reste la force **totale** dans les deux cas (2500N).

La table de correspondance.

Votre Phase A	Contrôleur Rapier
$k \cdot c \cdot L_{\text{repos}}$	<code>setWheelSuspensionStiffness</code>
$c_{\text{damp}} \cdot v_{\text{susp}}$	<code>setWheelSuspensionRelaxation / Compression</code>
plafond de course	<code>setWheelMaxSuspensionTravel</code>
μ latéral	<code>setWheelFrictionSlip</code>

force moteur maison	setWheelEngineForce
servo de lacet (M3)	setWheelSteering

Mêmes concepts, autres noms — c’est le but : vous savez déjà ce que **chaque paramètre fait**, parce que vous venez de l’écrire.

Partie 5 : Missions

Déjà fourni — ne pas réécrire

1. **M1 — Châssis** : boîte dynamique + 4 **raycasts visibles** (lignes) + meshes de roues. *Donné.*
2. **M4 — Terrain** : sol texturé, murs d’enceinte, rampe et bosses fixes. *Donné.*

Phase A — à implémenter

1. **M2 — Suspension** : ressort + amortisseur par roue (addForceAtPoint au point d’attache).
Check : la voiture se pose et se stabilise — elle n’oscille plus, elle ne traverse pas.
2. **M3 — Pilotage** : WASD/flèches — accélération, freinage, direction (addForceAtPoint **au sol**, groundPoint donné). *Check : on roule, on tourne, la caisse **pique** au freinage et **roule** en virage.*

Comptez 60-90 min de réglage — c’est normal, c’est le TP.

Phase B — l’outil du moteur

1. **M5 — Le contrôleur** : même véhicule, suspension remplacée par DynamicRayCastVehicleController (dans updateRapier). *Check : comportement au moins aussi bon, en ≈ 20 lignes.*
2. **M6 — Comparaison** : remplissez le tableau ci-dessous en roulant le **même parcours** dans les deux modes.

Objectif : reconnaître chaque paramètre — si vous ne savez pas ce que fait suspensionRelaxation, relisez votre Phase A.

M6 — tableau de comparaison (à remplir).

Critère	Phase A (maison)	Phase B (Rapier)
Lignes de code		
Stabilité (rampe, bosses, empilements)		
Réglage — combien de paramètres à toucher ?		
Feeling (précision, dérapage, toupie)		
Ce que je comprends / ce qui redevient une boîte noire		

Défi — le circuit

Servez-vous de la rampe et des bosses existantes comme circuit : rampe, virage serré, chicane.

Contrainte : ne pas se retourner, ne pas dérailler. Le vainqueur du grip (Phase A) ou du frictionSlip (Phase B) le plus **bas** sans sortir de piste gagne le respect éternel de la classe.

Bonus : ajoutez un **chrono** au HUD — départ au premier appui sur W, arrêt sur une ligne d'arrivée que vous placez.

💡 Le fichier solution

examples/session17_vehicle_solution.js — comme toujours : **essayez d'abord**. La Phase A est courte à écrire (≈ 20 lignes) mais longue à régler — c'est en réglant qu'on apprend.